

MEMORIA VIRTUALE E PAGE FAULT

Ogni processo ha uno spazio di indirizzamento virtuale, quindi utilizza INDIRIZZI VIRTUALI. La CPU genera indirizzi virtuali che vengono tradotti in indirizzi fisici (in RAM) dalla MMU.

INDIRIZZO VIRTUALE: VPN | offset

VPN → numero della pag virtuale

Offset → posizione del dato all'interno della pagina.

INDIRIZZO FISICO: PFN | OFFSET

La MMU usa come chiave di ricerca nella PAGE TABLE il VPN, per trovare il corrispondente PFN (frame fisico): VPN → PFN. L'offset rimane invariato.

Con d costante (offset), non cambia la dimensione della pag, quindi non cambia il numero di PF.

PAGE FAULT = eccezione generata da hw (MMU) quando un processo tenta di accedere ad una pagina virtuale che non è presente in RAM. (pag non residente).

Il page fault non è un errore del programma!!!

Come viene gestito un PF?

Viene eseguita un'istruzione di lettura/scrittura, che richiede l'accesso in memoria. La CPU genera l'indirizzo virtuale corrispondente e lo invia alla MMU che è responsabile della traduzione in indirizzo fisico. La MMU consulta la PAGE TABLE e a questo punto ci sono 2 possibilità:

- PRESENT BIT = 1 → contiene la traduzione e si può recuperare subito il frame fisico
- PRESENT BIT = 0 → CPU genera una trap (eccezione hw, page fault exception), perché l'evento richiede l'intervento del SO. CPU quindi interrompe l'istruzione corrente, salva il contesto del processo e passa da USER MODE A KERNEL MODE. Poi trasferisce il controllo alla routine chiamata PAGE FAULT HANDLER. Supponiamo che la pag si trovi nel backing store (nella SWAP AREA), con il page fault handler si va a verificare se l'indirizzo richiesto appartiene allo spazio di indirizzamento del processo e si recupera la pag da disco. Qualora l'indirizzo fosse illegale il processo terminerebbe immediatamente (SEGMENTATION FAULT). SO trasferisce a questo punto la pag da backing store (disco) alla RAM, cerca un frame libero. Se la RAM è piena cerca una PAG VITTIMA adottando un algoritmo di sostituzione (PAGE REPLACEMENT). Se la pag vittima ha DIRTY BIT = 1, prima la salva sul disco e poi carica la nuova pag sul frame disponibile (nel frattempo il processo viene messo nello stato BLOCKED), altrimenti se DIRTY BIT = 0, non serve salvarla. Aggiorna la PAGE TABLE, setta PRESENT BIT = 1, inserisce il numero del frame fisico, aggiorna i bit di stato ed aggiorna eventualmente anche

la TLB. Il contesto del processo viene ripristinato, la CPU ritorna un USER MODE e l'istruzione che aveva causato PF viene rieseguita.

Quindi PRESENT BIT = 0 può essere per diversi motivi:

- Pag mai caricata in RAM (DEMAND PAGING = SO carica le pag in RAM solo quando vengono effettivamente referenziate)
- Pag spostata sul disco (SWAPPED OUT)
- Indirizzo illegale, non appartiene allo spazio di indirizzamento del processo.

- Caratteristica	Dimensione pagina preferibile	Perché
Frammentazione interna	Minore	L'ultima pagina contiene mediamente meno spazio inutilizzato.
Dimensione della Page Table	Maggiore	Pagine più grandi $\Rightarrow$ meno pagine virtuali $\Rightarrow$ meno entry nella page table.
Sovraccarico I/O	Maggiore	Si trasferiscono più dati con una singola operazione, ammortizzando il costo fisso dell'I/O.
Numero di page fault	Maggiore	Ogni page fault carica più dati contigui, che potrebbero essere usati successivamente grazie alla località spaziale.
Località	Minore	Permette di caricare più precisamente solo le zone effettivamente utilizzate, evitando dati inutili.
Copertura della TLB	Maggiore	A parità di entry, ogni entry copre più byte di memoria virtuale.

LOCALITA'=La località significa che un processo, in un certo intervallo di tempo, utilizza soltanto una piccola zona del proprio spazio di indirizzamento. Con pagine piccole, il sistema carica in RAM in modo più preciso solo la zona effettivamente utilizzata.

COPY-ON-WRITE (COW): tecnica usata principalmente durante la creazione di un processo con la fork(). Inizialmente, dopo la fork(), padre e figlio condividono gli stessi frame fisici, le pag sono READ-ONLY (protette in sola lettura). Quando uno dei 2 processi tenta di scrivere / modificare una di queste pag si verifica un FAULT DI PROTEZIONE e il SO copia quella pag in un nuovo frame così che il processo possa modificare la propria copia.

VANTAGGI:

- Tempo: fork() + rapido perché non copia subito tutte le pag.
- Memoria: vengono copiate solo le pag effettivamente modificate.
I vantaggi dipendono da quante pag vengono scritte. Se il figlio esegue subito exec() allora il vantaggio è molto elevato

SWAP AREA: Domanda: Perché l'I/O sulla swap è più veloce dell'I/O sul file system? Lo spazio virtuale può essere maggiore della swap? La swap è più veloce perché viene gestita come uno spazio dedicato alle pag, senza dover gestire i nomi, directory, FCB, inode, permessi.... Il SO deve solo associare una pagina alla sua posizione nella swap

area. Lo spazio virtuale può essere maggiore della swap area perché non tutte le pag devono essere salvate lì, alcune non sono mai utilizzate, altre sono in RAM...

DOMANDE TEORICHE:

- **Una tabella delle pagine può adattare dinamicamente la propria dimensione in base alle pagine utilizzate dal processo? SI** → Con strutture come le TABELLE DELLE PAG GERARCHICHE (allochiamo livelli inferiori solo quando servono → riduciamo spreco di memoria).
- **Se un processo possiede più frame fisici rispetto alla dimensione del suo working set, il tasso di page fault diventa sempre zero? NO** → Avere abbastanza frame per contenere il working set corrente riduce i page fault, ma non garantisce che siano sempre zero. Il working set può cambiare e il processo può accedere a nuove pagine non ancora presenti in memoria.
- **Due indirizzi virtuali differenti possono essere mappati sullo stesso indirizzo fisico? SI** → Due indirizzi virtuali possono essere associati allo stesso frame fisico. Questo avviene, per esempio, con la **memoria condivisa**.
- **Pagine di dimensione maggiore possono rendere più efficiente l'utilizzo della TLB, nonostante aumentino la frammentazione interna? SI** → Con pagine più grandi, ogni entry della TLB permette di mappare una quantità maggiore di memoria. Servono quindi meno entry per coprire lo stesso spazio di indirizzamento e si possono ridurre i TLB miss. Tuttavia, pagine più grandi possono aumentare la frammentazione interna.
- **La presenza di una TLB garantisce sempre un tempo di accesso alla memoria inferiore rispetto a un sistema senza TLB? NO** → La TLB velocizza l'accesso quando si verifica un **TLB hit**. In caso di **TLB miss**, invece, è necessario consultare la tabella delle pagine, introducendo un costo aggiuntivo. Quindi NON SEMPRE.
- **L'algoritmo LRU garantisce il minor numero possibile di page fault? NO** → Quello è l'algoritmo ottimo che non è implementabile.
- **Il prepaging è vantaggioso solo se la probabilità di utilizzo supera l'80%? NO** → **non c'è alcuna percentuale di riferimento**. Convieni se molte delle pag caricate anticipatamente vengono poi effettivamente usate.
- **Tutte le strutture del kernel richiedono un'allocazione contigua? NO**
- **La frammentazione aumenta con pag + grandi perché servono partizioni contigue più grandi? NO** → Aumenta la frammentazione interna ma non perché servono partizioni contigue più grandi, infatti nel pagin le pag virtuali possono stare anche in frame fisici non contigui.
- **Lo slab allocator usa esclusivamente dimensioni che sono potenze di 2? NO** → Crea cache di oggetti della dimensione richiesta dalle strutture del kernel, non necessariamente potenze di 2 (quello è il buddy system).

- Una free list di pag ha una frammentazione interna media pari a metà pag?
NO, non ha senso parlare di frammentazione interna visto che non ci sono pag allocate ma frame liberi!
- Per i BUDDY SYSTEM: La frammentazione esterna viene eliminata allocando blocchi di dimensione multipla di una pagina. — FALSO
- Con Buddy non è possibile alcuna frammentazione interna. — FALSO
- La frammentazione esterna viene eliminata; è possibile soltanto quella interna. — FALSO → riduce molto la frammentazione ESTERNA ma non la elimina
- I blocchi liberi possono avere soltanto dimensioni pari a una potenza di 2, riducendo la frammentazione esterna. — VERO

RAID E AFFIDABILITA' DELLO STORAGE

RAID = Redundant Array of Independent Disks → storage technology che combina + dischi fisici e li presenta al SO come una sola unità logica. Abbiamo + livelli di RAID.

3 TECNICHE :

- **STRIPING** → distribuzione dei dati su + dischi
- **MIRRORING** → duplicazione completa dei dati
- **PARITY** → informazioni di ridondanza aggiuntive che permettono di ricostruire i dati

RAID 1 → MIRRORING → ogni dato viene scritto contemporaneamente su 2 dischi (entrambi contengono una copia IDENTICA dei dati).

Disco A	Disco B
_____	_____
Blocco X	Blocco X
Blocco Y	Blocco Y
Blocco Z	Blocco Z

Con 4 dischi → 2 coppie mirror.

Per avere PERDITA DI DATI effettiva si devono effettuare 2 guasti, il secondo guasto prima della riparazione del primo guasto.

Per AFFIDABILITA' (misura quanto è improbabile che il sistema perda definitivamente i dati).

MTTF = Mean Time To Failure → tempo medio di funzionamento di un componente PRIMA del guasto (dall'inizio di un'operazione al guasto). Si riferisce al singolo disco.

> MTTF → < probabilità di guasto, > affidabilità del componente.

MTTR = Mean Time To Repair → tempo medio necessario per riportare il sistema nello stato operativo dopo un guasto (individuazione del disco guasto, sostituzione del disco, ricostruzione dei dati sul nuovo disco).

<MTTR→ > affidabilità del RAID

MTTDL = Mean Time To Data Loss → tempo medio prima che il sistema perda definitivamente i dati. La perdita dei dati in RAID 1 ad es avviene quando falliscono entrambi i dischi della stessa coppia prima della ricostruzione.

>MTTDL→ > AFFIDABILITA'

Per avere affidabilità: > MTTDL, > MTTF, <MTTR

DOMANDE TEORICHE:

- **Una configurazione RAID 0 può migliorare l'affidabilità rispetto a un singolo HDD? NO** → Usa striping non ridondanza, quindi aumenta solo il throughput.
- **Un SSD può presentare problemi di frammentazione simili a quelli di un HDD? NO** → Con frammentazione si intende che i blocchi appartenenti allo stesso file non sono memorizzati consecutivamente sul dispositivo: 4 → blocco 80 → blocco 15 → blocco 120. Su un **HDD**, la frammentazione rallenta la lettura perché la testina deve spostarsi fisicamente tra zone diverse del disco. Su un **SSD**, invece, non esiste una testina: il controller accede elettronicamente alle celle. Quindi la posizione fisica non incide nello stesso modo e la frammentazione è molto meno problematica. Può comunque esistere FRAMMENTAZIONE LOGICA (blocchi del file sparsi, aumenta n. Operazioni I/O).
- **Uno scheduler di I/O ben progettato può migliorare significativamente le prestazioni di RAID 5? SI** → In RAID 5 le scritture coinvolgono il calcolo e l'aggiornamento della parità, quindi riordinare le richieste di lettura / scrittura, andando ad aggregare alcune operazioni può essere particolarmente utile.
- **Il mirroring può produrre un MTTDL inferiore all'MTTF? NO**: Con il mirroring se si rompe un disco, i dati non vengono persi perché sono ancora presenti nel disco B. La perdita di dati si verifica se si rompe il primo disco e poi durante la riparazione si rompe anche il secondo disco. MTTR di solito è <<MTTF. MTTDL quindi sarà molto grande. MTTDL>> MTTF.
- **Lo striping può ridurre l'MTTF di un singolo disco? NO**: divide i dati tra + dischi, quindi non modifica l'affidabilità (MTTF) di ciascun disco.
- **Dopo un guasto, l'utilizzo del sistema deve essere interrotto fino al completamento della riparazione. — FALSO** (basta avere ridondanza e continua a funzionare)
- **Con dischi in striping aventi lo stesso MTTF, la probabilità di guasto diminuisce aumentando . — FALSO** → aumentando N, diminuisce MTTDL e quindi aumenta la probabilità di guasto.
- **A ha 2 dischi in mirroring; B ha 4 dischi in striping + mirroring. L'MTTDL di B è superiore a quello di A. — FALSO** → in B ci sono + componenti che possono guastarsi.
- **Il mirroring include il bit di parità → FALSO** (copia completa di dati è)

GESTIONE DELLO SPAZIO LIBERO

Sia la memoria (frame) che il disco (file system → blocchi) contengono frame / blocchi liberi e occupati.

2 POSSIBILITA':

- **LINKED LIST** → lista concatenata di blocchi liberi. Ogni nodo è un blocco libero e contiene un puntatore al blocco successivo. Si chiama FREE LIST anche.
 - VANTAGGI: ottimo per estrarre un singolo blocco libero (estrazione in testa, complessità $O(1)$). Ogni volta che viene liberato un frame viene riinserito in questa lista, in coda, quindi serve pochissima memoria aggiuntiva (allocazione / deallocazione veloci).
 - SVANTAGGIO: non efficiente con ricerca di blocchi liberi consecutivi perché non conosce la posizione dei blocchi e dovrei scorrere tutta la lista ($O(n)$).
- **BITMAP** → vettore di bit in cui ogni bit rappresenta un blocco. Solitamente 1 = libero, 0 = occupato. Frame: 0 1 2 3 4 5 6 7
Bitmap: 1 0 1 1 0 0 1 1
Per trovare un blocco libero basta cercare gli 1.
 - VANTAGGI: ricerca efficiente, semplice da analizzare, ottimo per cercare BLOCCHI CONSECUTIVI (basta cercare sequenza di 1). Infatti rappresenta l'intero spazio di memoria.
 - SVANTAGGI: occupa memoria

DOMANDE TIPICHE:

- Bitmap sono migliori delle linked list per cercare per INTERVALLI di blocchi/ frame contigui → VERO.
- In un sistema con PAGING, la FREE LIST rappresenta la soluzione migliore sia per la memoria dei processi (USER MEMORY) che per la memoria KERNEL → FALSO.
- I blocchi liberi di un file system basato su I-NODE sono gestiti meglio usando la bitmap → FALSO
- La ricerca di un blocco libero in una linked list ha un costo lineare in un sistema con gestione della memoria basata su una page table a 2 livelli → FALSO.
- Una linked free list ordinata ha un costo lineare sia per allocazione di memoria contigua che per la liberazione → VERO
- Una bitmap, pur richiedendo memoria aggiuntiva, offre prestazioni migliori in un file system basato su FAT (File Allocation Table) → FALSO.
- Durante la lettura dell'intero file dall'inizio alla fine, l'allocazione basata su linked list sarà probabilmente più efficiente dell'inode in termini di tempo di esecuzione, grazie al DMA? SI. In generale non si può dire, ma qui il DMA fa la differenza.

- Durante la lettura dell'intero file dall'inizio alla fine, l'allocazione collegata sarà più efficiente rispetto agli altri formati, contigua e inode, perché l'elenco collegato è particolarmente efficace con l'accesso sequenziale? NO
- La dimensione di un blocco del disco è la stessa su tutti e tre i file system. Possiamo supporre che la frammentazione interna sia la stessa su tutti i file system? NO
- L'allocazione contigua e l'inode garantiscono l'accesso casuale/diretto a tempo costante ? SI.

FREE LIST ORDINATA: il sistema può semplicemente scorrerla solo una volta e controllare se 2 indirizzi differiscono esattamente di una pag. Il costo è lineare ($O(n)$). Nel caso peggiore dobbiamo scorrere tutti gli n frame.

Si può ordinare per:

- SOLO PER INDIRIZZI FISICI → VERO (eventualmente contigui così facciamo una sola scansione)
- PER DIMENSIONE CRESCENTE → FALSO → i frame hanno la stessa dimensione
- PER INDIRIZZI FISICI E LOCIGI → FALSO → i frame usano indirizzi fisici
- ORDINAMENTO NON NECESSARIO → FALSO → aumenterebbe il costo di scansione.

COME È ORGANIZZATA LA MEMORIA PER LE VARIE STRUTTURE DATI:

TIPO STRUTTURA	Descrizione	Tipo di gestione free space
Memoria user (PAGING)	Lo spazio di indirizzamento di un processo è diviso in PAGINE VIRTUALI che possono essere allocate in QUALSIASI frame fisico (sparsi), perché tanto c'è la PT che mantiene la corrispondenza tra frame fisici e pag virtuali.	NO frame fisici CONTIGUI → meglio FREE LIST.
Memoria Kernel (PAGING)	Il kernel richiede l'allocazione di strutture dati che devono essere fisicamente contigue, tipo PT.	SI frame fisici CONTIGUI → meglio BITMAP
File system basato su INODE	I-node è una struttura dati che descrive il file (metadati + indirizzi blocchi dati su cui è memorizzato il contenuto del file, non contiene il contenuto del file). Il file viene ricostruito seguendo gli INDIRIZZI dei blocchi, non la loro posizione fisica. Quindi i blocchi NON devono essere contigui, possono trovarsi in qualsiasi posizione del disco.	NO blocchi CONTIGUI → meglio FREE LIST.
Memoria con PT a 2 livelli	Anche se la struttura della PT è gerarchica non cambia niente dal primo caso. La PT contiene traduzioni VPN → PFN, e le pag possono essere allocate in qualsiasi frame fisico, non per forza contigui.	NO frame contigui → meglio FREE LIST. La ricerca non è lineare ($O(n)$), basta estrarre in testa ($O(1)$).
FREE LIST NON ORDINATA	Allocazione /liberazione = $O(1)$ → basta estrarre in testa	

FREE LIST ORDINATA	Allocazione / liberazione = $O(n)$ costo lineare. Necessita inserire in una lista ordinata, quindi dobbiamo fare una ricerca nella lista, scorrendola tutta.	
File System basato su FAT	FAT è una tabella in cui ogni voce corrisponde ad un blocco / cluster del disco. I blocchi non devono essere contigui perché il file viene ricostruito direttamente seguendo i puntatori contenuti nella FAT. La free list può essere implementata direttamente nella FAT (tenendo info su cluster liberi).	NO blocchi CONTIGUI → FREE LIST.
File System con Allocazione Contigua	Ogni file occupa blocchi consecutivi sul disco, quindi servono degli intervalli contigui liberi	SI blocchi CONTIGUI → meglio BITMAP
File System con Allocazione concatenata	Ogni blocco del file contiene un puntatore al blocco successivo, quindi basta trovare un singolo blocco libero	NO Blocchi CONTIGUI → meglio Free list.
File System con Allocazione Indicizzata	Ogni blocco indice contiene gli indirizzi dei blocchi dati.	NO blocchi CONTIGUI → meglio FREE LIST.
File System basato su EXTENT	Un extent descrive un intervallo contiguo (start block, length)	SI Blocchi contigui → meglio BITMAP
MEMORIA VIRTUALE SEGMENTATA	I segmenti sono regioni di dimensione variabile come codice, dati, stack. Ogni segmento deve occupare un intervallo contiguo di indirizzi fisici (memoria fisica).	SI frame CONTIGUI → meglio BITMAP
COUNTING	Invece di memorizzare ogni blocco libero separatamente, conserva per ogni intervallo: (primo blocco, numero di blocchi consecutivi liberi). Conviene quando esistono lunghi intervalli contigui e dobbiamo trovare un blocco abbastanza grande	SI blocchi contigui → il COUNTING è adatto alla ricerca di blocchi contigui.
GROUPING	Un blocco libero contiene gli indirizzi di molti altri blocchi liberi. L'ultimo indirizzo punta ad un altro blocco contenente un nuovo gruppo. E' una versione più efficiente della linked free list perché permette di conoscere molti blocchi liberi leggendo un solo blocco. NON è molto adatto a cercare CONTIGUITA' perché gli indirizzi elencati possono essere distanti	Il grouping riduce il numero di accessi al disco rispetto a una semplice linked free list.
LISTA DI INTERVALLI LIBERI	Ogni nodo rappresenta un intervallo (start, length). E' migliore di una lista di singoli blocchi per le allocazioni CONTIGUE. Se ordinata per INDIRIZZO FISICO facilita il COALESCING (fusione di intervalli adiacenti)	Una lista di intervalli ordinata per indirizzo fisico facilita la fusione dei blocchi liberati.
BUDDY SYSTEM	Memoria organizzata in blocchi di dimensione pari a potenze di 2: 1,2,4,8... pag. Se serve un blocco + piccolo, un blocco grande viene diviso in 2 BUDDY. Quando entrambi vengono liberati possono essere fusi insieme	Il buddy system permette una fusione efficiente dei blocchi liberi. Molto usato per allocazioni di memoria fisica contigua del kernel
SLAB ALLOCATOR	Gestisce oggetti kernel di dimensione nota (struct thread, proc, fd, inode)... Contengono oggetti già inizializzati e facilmente riutilizzabile.	Lo slab allocator è particolarmente adatto ad allocare oggetti kernel ripetuti della stessa dimensione.

ALBERI INDICIZZATI	Ricerca e inserimento hanno costo $O(\log(n))$. Particolarmente utili per gestire molti intervalli e cercare rapidamente quello adatto.	Un albero bilanciato può trovare un intervallo libero più efficientemente di una lista lineare.
--------------------	--	---

Tutte queste affermazioni non sono in senso assoluto! Si parla di cosa è meglio o + efficiente.

Una free list può essere ordinata per :

- Indirizzo FISICO: facilità contiguità + coalescing (fusione intervalli adiacenti)
- DIMENSIONE: facilità politiche tipo best fit
- NON ORDINATA: rende molto rapidi inserimento / rimozione di un solo elemento.

DOMANDE:

- Ordinare per indirizzo fisico facilita il coalescing. VERO
- Ordinare per dimensione facilita la ricerca degli intervalli fisicamente adiacenti. FALSO → facilità best fit
- Il buddy system elimina la frammentazione interna. FALSO
- Lo slab allocator sostituisce completamente il buddy system. FALSO → spesso lo slab allocator ottiene le pagine dal buddy system e poi le divide in oggetti.

STRUTTURE DELLE PAGE TABLE

PAGE TABLE STANDARD:

INDICE = VPN

Dimensione complessiva dipende da: dimensione spazio di indirizzamento virtuale processo, dimensione entry, numero di processi.

OGNI processo ha la sua PT.

Viene utilizzata per effettuare la traduzione VPN → PFN (traduzione diretta, costo accesso $O(1)$). Ogni entry specifica in quale frame fisico si trova la pag virtuale.

E' possibile realizzare una PT unica per tutti i processi → si tratta della IPT.

INVERTED PAGE TABLE (IPT):

Tabella GLOBALE per tutto il sistema, condivisa da tutti i processi: la stessa pag virtuale può comparire in processi diversi. Per questo ogni entry deve contenere anche il PID. Questo problema non c'è nella PT standard essendo unica per ogni processo.

INDICE = PFN (frame fisico)

CHIAVE DI RICERCA: (PID, VPN).

ENTRY IPT = PID, VPN, protection bit, valid, dirty.

Dimensione complessiva dipende da: numero di frame fisici (una entry per ogni frame fisico), quindi dipende dalla dimensione della RAM.

Frame (indice IPT)	Contenuto
0	PID=2, Pagina=8
1	PID=5, Pagina=10
2	PID=1, Pagina=3
3	PID=4, Pagina=25
4	PID=2, Pagina=7

Quando arriva un indirizzo virtuale, il sistema deve cercare un match (PID, VPN), quindi la ricerca è più costosa rispetto alla PT standard in cui abbiamo una traduzione diretta, perché non conosciamo il frame. Per trovare il match (PID, VPN) deve scorrere tutta la tabella, ed essendo costoso, per evitare di controllare tutte le righe, di solito insieme si usa una HASH TABLE per migliorare le performance.

DOMANDE:

- Con l'IPT si risparmia sempre memoria → FALSO: Non si risparmia memoria se abbiamo POCHI PROCESSI e uno SPAZIO DI INDIRIZZAMENTO PICCOLO. L'IPT è sempre dimensionata sulla base della RAM, la PT invece sullo spazio di indirizzamento (numero pag virtuali) e numero processi.
- In IPT risparmio di memoria dipende dalla dimensione della RAM → VERO. In generale il risparmio di memoria per entrambe dipende da numero processi, ampiezza spazio di indirizzamento, numero di frame.
- l'utilizzo di una tabella invertita riduce lo spazio di indirizzamento virtuale dei processi. FALSO
- in presenza di più processi, l'IPT è migliore in termini di memoria perché lo spazio di indirizzamento di ogni processo è almeno grande quanto la RAM. FALSO
- quando l'IPT viene combinata con una hash table, la dimensione raddoppia perché le entry dell'IPT vengono replicate nella tabella hash. FALSO
- se lo spazio di indirizzamento di un processo è pari alla metà della RAM, la PT del processo è generalmente più piccola dell'IPT. VERO
- la hash table velocizza la ricerca perché il numero di pagina viene utilizzato direttamente come indice nell'array hash. FALSO: il numero della pag virtuale è parte della chiave di ricerca, ma la chiave di ricerca completa è (PID, VPN). Poi la funzione hash calcola l'indice: $\text{indice} = h(\text{PID}, \text{VPN})$.
- risparmiamo sempre memoria con IPT se spazio virtuale processo > RAM → VERO: Se lo spazio virtuale è > della RAM, vuol dire che la PT sarà > della IPT,

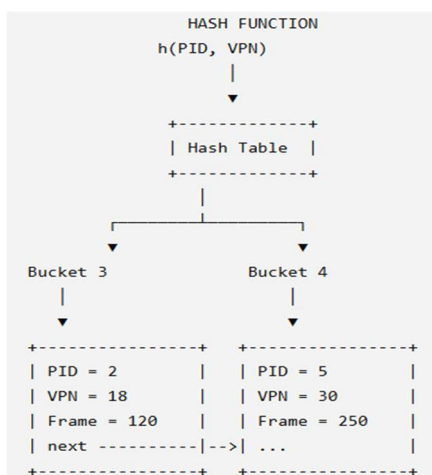
quindi risparmiamo memoria nell'usarla. Anche quando lo spazio di indirizzamento di ogni processo è $< \text{RAM}$, potremmo risparmiare memoria. Perché dobbiamo considerare la SOMMA degli spazi di indirizzamento, che comunque potrebbe superare la RAM.

HASHING

Alla IPT viene aggiunta una HASH table per migliorare le performance. Solitamente prende la coppia (PID, VPN) e la dà in pasto ad una funzione hash che produce un indice che identifica un BUCKET dove reperiamo la riga della IPT in cui trovare il frame corrispondente. E' una tabella aggiuntiva, non contiene la traduzione completa ma velocizza solo il tempo di accesso (Ricerca lineare in IPT troppo costosa), NON sostituisce l'IPT.

Hash	Punta a...
37	Riga 3 IPT
40	Riga 15 IPT
12	Riga 8 IPT

La soluzione IPT + HASH è diversa da una HASH PAGE TABLE. Nella prima soluzione l'IPT rimane la struttura principale di traduzione e l'hash è solo una struttura ausiliaria per velocizzare la ricerca. Nella seconda soluzione (HASH PAGE TABLE) invece costituisce una vera e propria struttura di traduzione che viene usata quando abbiamo uno SPAZIO DI INDIRIZZAMENTO MOLTO GRANDE (tipo architettura a 64 bit).



CHIAVE = (PID, VPN)

Quando la CPU genera un indirizzo virtuale (VPN + OFFSET), prendiamo la chiave (PID, VPN) e la diamo in pasto ad una funzione hash che restituisce il BUCKET della tabella hash. Ogni bucket contiene una linked list con VPN, PFN e puntatore al prossimo elemento (oppure VPN + PID + PFN + next). Se c'è un match tra il VPN iniziale e quello memorizzato nel bucket allora abbiamo trovato la traduzione, viene restituito il PFN che combinato con l'offset ci dà l'indirizzo fisico per accedere in RAM.

PROBLEMA: collisioni → se molte pag finiscono nello stesso bucket la lista diventa lunga e diventa quasi una ricerca lineare.

TLB

TLB = piccola memoria cache che memorizza le traduzioni + utilizzate. Prima la CPU controlla la TLB, se trova la traduzione (TLB HIT) allora evita l'accesso alla PAGE TABLE.

La TLB può contenere entry di + processi contemporaneamente? VERO

Abbiamo 2 possibilità:

- TLB CON **ASID** (Address Space Identifier) ogni entry ha anche questo campo ASID. Possono convivere entry di più processi diversi
- TLB **SENZA ASID**: non sa distinguere tra processi, quindi contiene solo le entry del processo corrente e deve essere svuotata (TLB FLUSH) ad ogni context switch per riempirla con le traduzioni del nuovo processo.

VALID BIT TLB

E' diverso da quello della PT. Se nella PT valid = 0 significa che la pag non è presente nella RAM, oppure la traduzione non è valida perché la pag virtuale non è associata ad un frame fisico valido. Invece nella TLB significa: la entry della cache è utilizzabile oppure no. Non dice niente sulla presenza della pag in RAM, ma sulla **validità della entry nella cache**.

- La copertura della TLB diminuisce quando aumenta la dimensione della pag?
NO → La copertura della TLB aumenta perché:
 $TLB \text{ REACH} = \text{numero entry} * \text{dimensione pag}$. Quindi aumentando la dimensione della pag aumenta anche la copertura.

DYNAMIC LOADING E DYNAMIC LINKING

- E' possibile caricare dinamicamente un programma (dynamic loading) senza che sia necessario il dynamic linking? SI →
DYNAMIC LOADING = il programma non viene caricato tutto all'avvio. Le funzioni vengono caricate solo quando servono per risparmiare memoria.

DYNAMIC LINKING = (come vengono risolti i riferimenti alle librerie), il programma non contiene il codice delle librerie, ma durante l'esecuzione vengono collegate librerie condivise (condividere codice tra + processi).

Sono 2 meccanismi indipendenti: posso avere dynamic loading ma tutto il programma è stato linkato staticamente, o viceversa.

Dynamic loading → ci dice quando caricare il codice in memoria (solo quando necessario)

Dynamic linking → collegamento delle librerie durante l'esecuzione

Il dynamic linking NON richiede che un programma sia anche caricabile dinamicamente /dynamic loading).

MEMORIE DI MASSA E GESTIONE DEL DISCO

DOMANDE TEORICHE:

- 1) Se aumentiamo il numero di dischi, diminuisce la probabilità che un disco si guasti? NO → Aumenta la probabilità che almeno uno si guasti. Supponiamo che ogni disco abbia probabilità di guasto p , la probabilità che almeno un disco si guasti tra n dischi è $1-(1-p)^n$. Questa cresce all'aumentare di n . Il RAID protegge i dati dal guasto di un disco con ridondanza, ma non riduce la probabilità che si verifichi un guasto.
- 2) SCAN e C-SCAN sono orientati agli SSD ? NO → sono degli algoritmi di scheduling utilizzati per minimizzare il tempo di latenza e gli spostamenti della testina, quindi orientati agli HDD che hanno parti meccaniche, invece gli SSD NO.
- 3) Nel multiboot, la partizione root deve contenere tutti i kernel: NO. Il multiboot è un computer con + sistemi operativi. All'accensione compare il bootloader (programma) che sceglie quale sistema avviare. Il bootloader DEVE SAPERE DOVE TROVARLI (in quali partizioni), ma non è necessario mettere tutti i kernel nella partizione root / del disco di avvio.
- 4) **Un'operazione su SSD presenta sempre latenza inferiore rispetto alla stessa operazione su HDD? SI** → Generalmente perché non hanno tempi di seek o latenza rotazionale, ma non sempre.
- 5) In un sistema con più dispositivi di I/O e una sola CPU, le operazioni di I/O possono avvenire in parallelo mentre la CPU è impegnata nell'esecuzione di un processo? SI. Ogni dispositivo ha il suo controller che esegue le operazioni di I/O indipendentemente dalla CPU che può eseguire altri processi. Al termine dell'operazione il controller invia interrupt a CPU per segnalare.
- 6) In un sistema con memoria virtuale demand-paging e DMA, può verificarsi un page fault durante un trasferimento DMA, causando il fallimento del trasferimento se non vengono adottate precauzioni? SI. Il DMA accede alla memoria fisica (usa indirizzi fisici) senza l'intervento continuo della CPU. Se il buffer usato dal DMA contenesse delle pag Virtuali non ancora caricate in RAM o sostituite durante il trasferimento avremmo PF che lui non può gestire autonomamente come farebbe la CPU e quindi il trasferimento potrebbe fallire. Per questo prima di avviare il DMA il SO deve caricare in RAM tutte le pag del buffer e bloccarle temporaneamente in memoria (pinning).
- 7) È possibile perdere un interrupt se il controller degli interrupt è impegnato a gestirne un altro e il dispositivo che ha generato il secondo interrupt non supporta l'accodamento? SI → Se supportasse accodamento, invece potrebbe
- 8) C-SCAN ottimizza il tempo di posizionamento negli HDD minimizzando la latenza rotazionale. FALSO: minimizza il seek time

- 9) dato lo stesso insieme di richieste, la distanza complessiva percorsa dalla testina in C-SCAN è generalmente maggiore rispetto a SCAN. VERO: C-SCAN torna da un'estremità all'altra senza servire richieste durante il ritorno; questo movimento aggiuntivo rende generalmente la distanza maggiore rispetto a SCAN.
- 10) memorie non volatili, come gli SSD, non richiedono gli stessi algoritmi di scheduling utilizzati per gli HDD. VERO
- 11) mentre SSTF sta servendo le richieste presenti nella coda, non possono essere aggiunte nuove richieste. FALSO: In SSTF viene selezionata la richiesta + vicina alla posizione corrente della testina. Quando completa una richiesta, controlla nuovamente la coda e sceglie la richiesta più vicina tra quelle presenti in quel momento. Quindi nel frattempo possono arrivare nuove richieste e vengono aggiunte alla coda.
- 12) la minimizzazione del seek time è l'obiettivo degli algoritmi di scheduling per HDD. VERO

Perché è utile spostare un blocco logico in SSD? Le celle flash di un SSD hanno un numero limitato di cicli di programmazione e cancellazione. Per effettuare una riscrittura dobbiamo prima effettuare una cancellazione, a livello di blocco fisico. Se un blocco logico fosse molto usato, sicuramente è alta la probabilità che raggiunga la usura. Con il WEAR LEVELING manteniamo una mappa: blocco logico $\rightarrow$ blocco fisico. Così possiamo distribuire le scritture e cancellazioni su blocchi fisici diversi modificando questa associazione e rendendo UNIFORME l'usura. Quindi il WEAR LEVELING non riduce il numero totale di scritture e cancellazioni (ci sono addirittura delle operazioni aggiuntive per lo spostamento dei dati), ma le ridistribuisce tra le celle. Così nessuna cella raggiunge troppo presto l'usura. L'SSD ciò nonostante può continuare anche con blocchi fisici inutilizzabili grazie al FLASH TRANSLATION LAYER, marca il blocco fisico come inutilizzabile, lo rimuove dalla mappa, sposta i dati validi in un blocco funzionante e associa il blocco logico ad un altro blocco fisico di riserva, appartenente ad uno spazio non visibile all'utente, detto overprovisioning.

SCAN e C-SCAN

Sono algoritmi di scheduling delle richieste di I/O. Vogliono ridurre seek time (tempo impiegato dalla testina del disco per spostarsi fino alla traccia contenente i dati richiesti). Riducendo gli spostamenti inutili della testina miglioriamo il tempo medio di risposta, la distanza complessiva percorsa dalla testina e il throughput del disco.

Si applicano principalmente agli HDD (dischi magnetici) che hanno componenti meccanici (testina mobile, traccia, piatti magnetici rotanti). La posizione fisica dei dati influenza il tempo di accesso. Non sono molto utili per NVM e SSD perché non hanno una testina meccanica.

I/O E DMA

DOMANDE TEORICHE:

- 1) L'I/O del disco viene generalmente gestito tramite DMA: SI → Senza DMA è la CPU a trasferire i dati dal controller del disco alla RAM. La CPU è occupata durante tutto il trasferimento e non può fare altre cose, spreca tempo. Con il DMA la CPU non trasferisce personalmente tutti i dati, ma si limita a configurare il controller del DMA, che esegue il trasferimento.
- 2) Cycle stealing: la CPU interrompe il DMA: NO → è il controller del DMA che temporaneamente utilizza alcuni cicli del bus di memoria che potrebbero essere altrimenti usati dalla CPU. Durante un ciclo, il DMA usa il bus e la CPU nel frattempo attende.
- 3) I/O interrupt è più efficiente dell'I/O basato sul polling? Dipende dall'op di I/O. → Per un dispositivo lento, gli interrupt sono generalmente più efficienti: la CPU non deve controllarlo continuamente. Per operazioni molto rapide e frequenti, invece, gestire continuamente gli interrupt può avere un costo maggiore rispetto a un breve polling. Quindi non è sempre più efficiente.
- 4) I/O INTERRUPT Può essere applicato all'I/O a blocchi? SI
- 5) Non può essere applicato all'I/O di caratteri → NO.
- 6) Può essere più efficiente per operazioni di I/O lente? Si. Con il polling controllerebbe continuamente lo stato inutilmente.
- 7) È necessario per implementare I/O asincrono? NO. Nell'I/O asincrono, il processo avvia un'operazione senza rimanere bloccato fino al completamento. Utile ma NON necessario (come parola è too much). Di solito si usa polling per questo.
- 8) «L'accesso a un byte del file può causare un page fault anche se non viene eseguita alcuna system call di I/O esplicita.» SI → si sta comunque tentando di accedere al file che deve essere caricato in RAM, quindi se non è caricato, PAGE FAULT. Non serve per forza chiamare una system call come read().
- 9) «Durante un trasferimento DMA verso un buffer utente, le pagine virtuali corrispondenti devono rimanere residenti in memoria fisica per tutta la durata del trasferimento.» SI → si parla di pag pinned (bloccate).
- 10) «Se una pagina coinvolta in un trasferimento DMA venisse spostata su disco, potrebbe verificarsi corruzione dei dati.» SI → DMA continuerebbe a leggere su un vecchio frame.
- 11) Un processo utente non può gestire direttamente un dispositivo tramite interrupt, ma, conoscendone l'indirizzo, può gestirlo in polling direttamente in modalità utente senza effettuare una chiamata di sistema. NO → Il fatto di conoscere l'indirizzo del dispositivo non è sufficiente. I registri dei dispositivi e le istruzioni di I/O sono normalmente protetti e accessibili soltanto in modalità kernel. Il processo dovrebbe entrare in modalità kernel per leggere il register status del dispositivo.

- 12) Quando un programma chiama `read()` su un file, la quantità di byte letti in una singola chiamata è limitata dalla dimensione dei blocchi del disco. NO → noi richiediamo `nbyte`, non ci interessa se supera la dimensione del blocco (competenza del SO).
- 13) Un file di testo con righe di lunghezza variabile può essere letto soltanto sequenzialmente, perché l'indirizzo logico di una determinata riga non è noto. NO → SERVONO STRUTTURE AGGIUNTIVE. Tuttavia, è possibile accedere direttamente a una riga se ne conosciamo l'offset nel file, per esempio grazie a un indice.
- 14) La durata in vita di un'unità SSD è limitata dal numero massimo di operazioni di cancellazione. SI → Prima di riscrivere una pagina, il blocco che la contiene deve essere cancellato. Le cancellazioni degradano progressivamente le celle. Per limitare il problema delle NAND Flash si usa garbage collection, rimappatura dei blocchi danneggiati, wear leveling, blocchi di riserva.
- 15) Quando una pagina di un SSD raggiunge il numero massimo di scritture, l'intero disco smette di funzionare e deve essere riparato. NO → Il guasto di una singola pagina non comporta immediatamente il guasto dell'intero SSD.
- 16) Un controller DMA utilizza indirizzi logici, perché il driver che programma il DMA non può conoscere gli indirizzi fisici. NO (USA indirizzi fisici, solo DVMA può usare indirizzi logici).
- 17) «Aumentare la dimensione della pagina può ridurre il numero di page fault negli accessi sequenziali, ma aumentare il costo negli accessi casuali.» SI perché sfruttiamo la località, ma rischiamo di sprecare memoria e non caricare le pag realmente utili.
- 18) «Quando un file viene letto sequenzialmente usando `read()`, il kernel può sovrapporre I/O e calcolo più facilmente rispetto all'accesso tramite memory-mapped file.» → NO: Con `read()` il programma richiede esplicitamente i dati, il kernel li legge dal disco spesso anticipando i blocchi successivi con il read-ahead mentre il processo elabora i dati già ricevuti, il kernel può caricare quelli successivi. Con `mmap()` il file viene associato allo spazio di indirizzamento del processo (mapped file), quando il processo accede ad una pag non residente avviene un PF e il kernel può caricare la pag e anticipare quelle successive con read-ahead. Quindi entrambi consentono di sovrapporre calcolo e I/O. Non è corretto dire che il kernel possa farlo più facilmente necessariamente con `read()`.
- 19) «Un page fault causato dall'accesso a un file mappato in memoria può bloccare il processo chiamante anche se l'I/O su disco necessario è svolto tramite DMA.» →
il DMA esegue il trasferimento senza occupare la CPU, ma il processo non può proseguire finché la pagina richiesta non arriva in RAM. Il processo viene quindi bloccato, mentre la CPU può eseguire altri processi.

20) . Il DMA è sempre più efficiente dell'I/O gestito mediante interrupt? NO→ è particolarmente efficiente per trasferimenti di grandi quantità di dati perché non coinvolge continuamente la CPU. Ma per trasferimenti piccoli e veloci il costo per configurare il DMA potrebbe essere troppo elevato e quindi conviene I/O mediante interrupt.

21) In una rete ad alta velocità, il polling è più efficiente degli interrupt? No→ Il polling richiede che la CPU controlli continuamente lo stato del dispositivo, consumando tempo di elaborazione anche quando non ci sono eventi. Gli interrupt permettono invece alla CPU di eseguire altre attività e di intervenire solamente quando il dispositivo segnala un evento.**Nota:** nei sistemi di rete ad altissime prestazioni il polling può essere utilizzato per ridurre il costo prodotto da un numero eccessivo di interrupt, ma non è la risposta prevista da questa soluzione.

22) Viene generato un interrupt per ogni carattere ricevuto oppure uno per ogni “invio”? (TASTIERA) viene normalmente generato un interrupt per ogni evento della tastiera, quindi anche la pressione del tasto Invio genera il proprio interrupt. Il programma può interpretare Invio come fine riga, ma questa interpretazione avviene a un livello superiore.

23) Per un dispositivo di I/O gestito a blocchi, una sola chiamata write() può richiedere la scrittura di più di un blocco? SI→ Una singola write() può trasferire una quantità di dati maggiore della dimensione di un blocco oppure iniziare in un blocco e terminare in quello successivo. Di conseguenza, può coinvolgere più blocchi del dispositivo.

24) Supponendo che il sistema utilizzi un controller DMA, un'operazione di lettura o scrittura inserita successivamente nella coda da un altro processo può essere eseguita prima di quella corrente? Vale lo stesso per richieste dello stesso processo? SI→ Il driver o lo scheduler del dispositivo può riordinare le richieste pendenti per migliorare le prestazioni, purché non vengano violate dipendenze o vincoli di ordinamento. Questo può avvenire sia tra processi diversi sia tra richieste indipendenti dello stesso processo. Le pagine coinvolte nel DMA vengono bloccate in RAM per impedirne lo spostamento o la sostituzione, ma ciò non impedisce al dispositivo di gestire o riordinare altre richieste. Con chiamate sincrone sequenziali dello stesso processo, invece, la seconda richiesta viene effettuata soltanto dopo il completamento della prima.

DISPOSITIVO DI I/O A BLOCCHI = trasferisce gruppi di dati (chiavette USB, SSD, HDD).

POLLING = POLLING = CPU controlla continuamente se il dispositivo è pronto, durante l'attesa spreca cicli di clock (busy wait)

INTERRUPT = CPU avvia l'operazione di I/O e contemporaneamente fa altro. Quando il dispositivo di I/O è pronto invia un interrupt.

Data-in / data-out register contengono i dati trasferiti, non il read-or-write bit che è un bit di controllo. Busy bit indica se il dispositivo è occupato o meno, sta nello status register.

Il command-ready bit sta nello status register e indica se il dispositivo è pronto a ricevere un comando e infine il read-or-write bit sta nel control register e specifica il tipo di operazione richiesta.

Il dispositivo può inviare l'interrupt direttamente al processo destinatario? NO → **No**. Il dispositivo invia l'interrupt alla CPU, che passa al kernel ed esegue l'interrupt handler del driver. Il kernel identifica poi il processo interessato e, se era bloccato in attesa dell'I/O, lo risveglia.

Cosa succede se write(fd, buf, nb) riceve un indirizzo buf in kseg0? La system call deve rifiutare l'indirizzo e restituire EFAULT, perché un processo utente non può indicare come buffer un indirizzo del kernel. SYS_write deve usare copyin oppure uiomove, che verificano che il puntatore appartenga allo user space. Dichiarare il parametro come userptr_t non basta: è soltanto un tipo, non una validazione dell'indirizzo.

I/O SINCRONO E ASINCRONO

In I/O sincrono bloccante, il processo richiede l'I/O e viene sospeso fino al completamento, poi ritorna la system call, ma la CPU nel frattempo può eseguire altri processi.

In I/O sincrono non bloccante: la chiamata ritorna immediatamente con ciò che è disponibile, se non ci sono dati restituisce ad es EAGAIN. Il processo controlla e riprova.

In I/O asincrono il processo avvia l'intera operazione e continua l'esecuzione. Il kernel in background mantiene attiva la richiesta e notifica successivamente il completamento con un evento, segnale o callback. L'aspetto essenziale è che **avvio della richiesta** e **completamento della richiesta** sono due momenti distinti.

Polling e interrupt descrivono come il driver comunica con il controller, mentre sincrono e asincrono descrivono ciò che osserva il processo. Un I/O sincrono può usare anche polling e interrupt. Anche I/O asincrono, anche se è più efficiente con interrupt, visto che permettono al kernel di essere avisato del completamento senza controllare ripetutamente il dispositivo. Con IO asincrono sovrapponiamo calcolo e trasferimento, quindi può lavorare mentre attende. Calcola senza usare i dati coinvolti nel trasferimento però. Crea concorrenza tra elaborazione e trasferimento, ma è difficile da usare perché richiede sincronizzazione.

DOMANDE TEORICHE:

Supponiamo che L'applicazione legge un file binario contenente numeri interi a 32 bit e ne calcola la somma. La lettura avviene in modalità asincrona.

- Per garantire che tutti i dati siano stati letti, è sufficiente che l'applicazione attenda il completamento dell'ultimo I/O, perché quando questo viene completato risultano completati anche tutti gli I/O precedenti? NO → Anche

- se l'ordine di avvio era $A \rightarrow B \rightarrow C$, l'ordine di completamento comunque potrebbe essere $C \rightarrow A \rightarrow B$, solo se ci sono dipendenze conta ordine
- Se il file viene letto utilizzando più thread, la somma deve essere considerata una sezione critica e quindi protetta con opportune primitive di sincronizzazione? SI
 - La modalità asincrona è implementata internamente tramite I/O basato su interrupt, perché il polling non può supportare la modalità asincrona? NO
L'asincrona supporta sia polling sia interrupt. L'interrupt è generalmente più efficiente, ma non è obbligatorio.
 - È possibile avviare letture multiple, ma nessun dato deve essere utilizzato finché l'I/O corrispondente non è stato completato. Possono essere attive più richieste contemporaneamente; • ogni buffer diventa utilizzabile soltanto dopo il completamento della propria richiesta.

FILE

L'applicazione può Vedere I file organizzati in diversi formati:

- SEQUENZA DI BYTE
- STRUTTURA A RECORD: possono avere lunghezza fissa o variabile (come righe che terminano con /n).
- STRUTTURA COMPLESSA: file organizzati in header e/o sezioni, perché contendono dati + metadati. I file ad indici sono un esempio. Abbiamo gli index file che contengono una tabella con entry (chiave, puntatore) per permettere una ricerca efficiente, in quanto il puntatore punta in modo diretto al dato nel relative file (file di dato).

Il formato o il tipo di file sono info di LIVELLO APPLICATIVO (gestite da app), mentre allocazione contigua, linked list, FAT, INODE, sono modalità con cui il file system organizza fisicamente o logicamente i blocchi. Il file system vede essenzialmente una sequenza di byte, indipendentemente dal suo contenuto.

Supponiamo di avere un file suddiviso in : HEADER + SECTIONS.

- L'applicazione può presumere che l'header e ciascuna sezione del file siano memorizzati a partire dai confini dei blocchi del disco nel file system? NO → L'applicazione vede il file come una sequenza di byte organizzati logicamente in header + sezioni, non vede come sono organizzati in blocchi fisici (l'allocazione è competenza del SO). L'app non può assumere che header + sezioni siano allineati ai confini dei blocchi di disco.
- L'applicazione può organizzare i dati in più file anziché in un unico file, utilizzando un file per l'header e un file per ogni sezione? In caso affermativo, come devono essere rappresentati i riferimenti alle sezioni contenuti nell'header? SI → Organizzandoli in file separati gli offset (rispetto all'inizio

del file unico) non sono più significativi. Nell'header ogni sezione può quindi essere identificata tramite il **nome o il percorso del file** che la contiene ed eventualmente tramite un offset interno a quel file.

- La dimensione dei blocchi del disco è uguale nei tre file system. Possiamo assumere che anche la frammentazione interna sia uguale? NO → Nell'allocazione concatenata, una parte di ogni blocco è utilizzata per memorizzare il puntatore al blocco successivo. Di conseguenza, ogni blocco può contenere meno byte di dati rispetto all'allocazione contigua e a quella tramite inode. Pertanto, la frammentazione interna non è necessariamente uguale.
 - I tre file di testo a.txt, b.txt e c.txt possono condividere lo stesso blocco del disco, cioè essere memorizzati almeno parzialmente nello stesso blocco? NO → Sono tre file distinti e il file system assegna a ciascuno un insieme separato di blocchi
 - Dopo aver riunito o compresso i tre file in un unico archivio, i loro contenuti possono condividere lo stesso blocco del disco? Si → **L'archivio è considerato dal file system come un unico file.**
 - **Il puntatore alla posizione corrente di lettura/scrittura di un file aperto risiede nel File Control Block. — FALSO** → FCB contiene: dimensione, proprietario, puntatori ai blocchi dati, permessi. L'offset corrente appartiene alla entry della SYSTEM-WIDE OPEN FILE TABLE
1. Il requisito principale per implementare le directory è una ricerca efficiente, con il nome del file come chiave. — VERO → Deve tradurre velocemente: nome file → inode/FC. Può essere implementata con liste, tabelle hash, alberi
 2. La close() può modificare sia la per-process file table sia la system-wide open-file table. — VERO. Perché elimina il riferimento al fd nella tabella del processo, quindi decrementa il contatore di riferimenti nella tabella globale. Se il contatore arriva a zero, libera l'open file globale.
 - PER-PROCESS FILE TABLE ALLOCATA NELLO SPAZIO USER → FALSO (sta nel kernel).
 - L'applicazione può presupporre che l'intestazione e ciascuna sezione siano allineate ai limiti dei blocchi del disco? NO
 - Se l'intestazione è memorizzata in un file separato, questa deve essere allocata in blocchi di dati contigui? NO → L'allocazione fisica dipende dal metodo usato dal file system.
 - È obbligatorio che l'applicazione organizzi i dati in più file: uno per l'intestazione e uno per ogni sezione? NO
 - Una volta conosciuto, dopo aver letto l'intestazione, l'offset di una sezione, è possibile leggere la sezione con accesso diretto/casuale con tutti i formati di allocazione dei file, oppure alcuni richiedono necessariamente accesso

sequenziale? SI con tutti i formati. L'APP se conosce l'offset può spostare la posizione corrente del file in questo modo:

```
lseek(fd, offset, SEEK_SET);
```

```
read(fd, buffer, dimensione);
```

➔ quindi non è obbligata a leggere/utilizzare tutti i byte precedenti

TIPO FILE	ACCESSO
Sequenza di byte	SEQUENZIALE + DIRETTO
Record lunghezza FISSA	SEQUENZIALE + DIRETTO
Record lunghezza VARIABILE	SEQUENZIALE
Struttura complessa	SEQUENZIALE + DIRETTO (ma dipende dal tipo di formato).

Se utilizziamo un algoritmo di ordinamento con complessità $O(N \log N)$:

Organizzazione	Complessità
Allocazione contigua	$O(N \log N)$
Linked allocation	$O(N^2 \log N)$
FAT	$O(N^2 \log N)$, ma con costante migliore
Inode	$O(N \log N)$

Per stampare sequenzialmente N record di un file:

Organizzazione	Complessità
Allocazione contigua	$O(N)$
Linked allocation	$O(N^2)$
FAT	$O(N^2)$, con costante migliore
Inode	$O(N)$

- La tabella dei file descriptor per processo si trova in memoria user o kernel? **MEMORIA KERNEL**
- Due processi chiamano `write(fd, V, N)` usando entrambi `fd = 5`. Scrivono entrambi sullo stesso file sicuramente? **NO** ➔ Ogni processo ha una tabella dei fd separata, quindi i due 5 possono indicare file differenti.
Non possono scrivere mai sullo stesso file ? **NO**, potrebbero
Generalmente scrivono su file diversi ma possono scrivere sullo stesso? **SI**
Per scrivere sullo stesso file devono usare fd diversi? **NO**.
- **Il formato a record di un file di lunghezza variabile può influenzare l'allocazione? NO.** Il formato del file riguarda l'applicazione, non il file system che vede il file come una semplice sequenza di byte e lo divide in blocchi. Non interpreta le righe e non gli interessa se hanno lunghezza

variabile o meno. Riempie i blocchi con un numero fisso di byte, non di righe, quindi possono iniziare in un blocco e terminare anche al successivo.

ACCESSI:

- L'accesso sequenziale può essere simulato mediante accesso diretto → VERO
- L'accesso diretto può essere implementato con formati di allocazione basati su liste, come FAT o lista concatenata
- L'accesso diretto può essere implementato da tutti i formati di allocazione dei file. — VERO (ma con costo diverso).
- **L'accesso diretto non può essere simulato mediante l'accesso sequenziale. — FALSO (si scorre il file fino alla posizione desiderata, anche se inefficiente).**

OS161

WCHAN_SLEEP()

- «Nei wait channel, il lock rilasciato prima dell'attesa e riacquisito alla fine è di tipo busy-wait.» si → **Il lock passato a wchan_sleep() è uno spinlock**, quindi quando un thread tenta di acquisirlo può fare busy-wait. **Il tipo di lock rilasciato e riacquisito è busy-wait.**
- **L'attesa sul wait channel non è busy-wait**: il thread viene sospeso e non usa la CPU. **Il modo in cui il thread attende nel wait channel non è busy-wait.**

Quindi:

- Attesa per acquisire lo SPINLOCK → può essere BUSY WAIT
- **attesa dell'evento dentro il wait channel** → è blocking, non busy-wait.

DUMBVM (ALLOCAZIONE DELLA RAM)

- kalloc alloca in una partizione fisica distinta da quella dei processi user? NO → In dumbvm non esistono due partizioni fisiche fisse: memoria kernel e user provengono dalla stessa RAM fisica.
- as_prepare_load usa ram_stealmem, che non permette di liberare memoria; per implementare ram_freemem bisogna modificare tale schema? SI → Bisogna introdurre strutture che registrino le allocazioni, come bitmap e dimensione dei blocchi, così da poter restituire i frame.
- ram_stealmem restituisce un puntatore logico perché il software non può usare indirizzi fisici? NO → Restituisce un paddr_t, cioè un indirizzo fisico. Successivamente, quando necessario, viene convertito in indirizzo virtuale kernel

- ram_stealmem alloca memoria incrementalmente dal primo indirizzo disponibile dopo il kernel? SI→ Utilizza un puntatore al primo indirizzo fisico libero e lo fa avanzare a ogni allocazione.
- ram_stealmem impedisce che allocazioni kernel e user siano intercalate? NO→ Non crea partizioni separate: assegna memoria progressivamente secondo l'ordine delle richieste, che possono provenire sia dal kernel sia dai processi.
- ram_stealmem viene usata sia per la memoria kernel sia per quella user? SI→ In dumbvm è alla base sia di alloc_kpages/kmalloc, sia dell'allocazione delle pagine dell'address space user.
- Poiché dumbvm usa allocazione contigua, la TLB è disabilitata e si usano registri base/limite? OS/161 usa comunque paginazione e TLB. La contiguità fisica dei segmenti in dumbvm non trasforma il sistema in uno basato su registri base/limite.
- Con una bitmap serve un secondo array che registri la dimensione delle allocazioni? SI→se la funzione di liberazione riceve soltanto l'indirizzo iniziale. L'array permette di sapere quanti frame consecutivi liberare, tanto per allocazioni kernel quanto user.
- Lo stack user viene allocato alla fine della RAM fisica, procedendo a ritroso?NO→È collocato alla fine dello spazio virtuale user, presso USERSTACK. I suoi frame fisici vengono assegnati dall'allocatore come gli altri.
- Lo stack kernel del processo è una struttura globale del kernel? NO→ Ogni thread possiede il proprio kernel stack; non è uno stack globale condiviso.
- Allocare memoria in multipli di pagina riduce la frammentazione interna? NO, l'aumenta.
- La memoria viene allocata in pag perché la MMU MIPS usa una TLB? SI
- Dumbvm implementa una tabella delle pag→ si
- Kmalloc può allocare solamente quantità di memoria multiple della dimensione di una pag NO
- Non è necessaria la ricerca di frame liberi contigui, perché MIPS possiede una TLB. NO
- La free list dovrebbe essere utilizzata soltanto per il segmento di codice e il segmento dati, non per lo stack utente, perché in dumbvm lo stack ha

dimensione fissa. NO→ La dimensione fissa dello stack non significa che lo stack non abbia bisogno di memoria fisica

- La free list può essere organizzata efficacemente come una lista di intervalli contigui di frame liberi. SI
- Occorre supportare la ricerca di intervalli di frame liberi contigui. SI→ servono contigui perché in `dumbvm getppages()` restituisce un solo indirizzo fisico, quello del primo frame. Non basta contare quanti frame sono liberi in totale: bisogna trovare `npages` frame liberi consecutivi.
- `ram_stealmem` alloca progressivamente la memoria a partire dal primo indirizzo disponibile dopo il caricamento del kernel. VERO→ `ram_stealmem` è un BUMP ALLOCATOR: mantiene l'indirizzo del primo frame libero e ad ogni richiesta lo sposta in avanti.
- `ram_stealmem` viene utilizzata per allocare memoria dinamica del kernel e memoria dei processi utente. VERO
- per implementare `freeppages`, al posto della bitmap si potrebbe utilizzare una lista concatenata. VERO
- usando una bitmap è sempre necessario un secondo array contenente le dimensioni delle partizioni allocate, sia per il kernel sia per i processi utente. FALSO→ serve numero frame da liberare. Per i processi utente le dimensioni sono già memorizzate nell'addresspace. Con kernel, quando chiamiamo `free_kpages` normalmente si passa solo l'indirizzo iniziale, quindi in questo caso è necessario conservare da qualche parte la dimensione dell'allocazione per sapere quanti frame liberare, ma non valido per entrambi.
- `free_kpages`, chiamata da `kfree`, può liberare lo stack kernel di un processo. — VERO
- `vaddr_t` può rappresentare indirizzi logici kernel e user. — VERO
- `paddr_t` può essere usato per indirizzi logici kernel e user. — FALSO
- `as_prepare_load()` restituisce la dimensione dello spazio di indirizzamento in numero di pagine. — FALSO → restituisce un codice di errore (0 se successo, `ENOMEM` se errore).
- `getppages()` chiama `ram_stealmem()`. — VERO

PRIMITIVE DI SINCRONIZZAZIONE

- Perché in un contesto multi-core (+CPU) non è possibile realizzare mutua esclusione semplicemente disabilitando e riabilitando l'interrupt?

La mutua esclusione mediante disabilitazione dell'interrupt si basa sull'assunzione che la concorrenza possa essere realizzata unicamente interrompendo il processo/thread attualmente in esecuzione. Questo è possibile solo in un sistema single-core. In un multi-core disabilitare l'interrupt su una CPU (core) non previene l'esecuzione di un altro processo/thread su un'altra CPU (core).

P() E V()

```
void P(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    while (sem->sem_count == 0) {
        wchan_sleep(sem->sem_wchan, &sem->sem_lock);
    }
    sem->sem_count--;
    spinlock_release(&sem->sem_lock);
}

void V(struct semaphore *sem) {
    spinlock_acquire(&sem->sem_lock);
    sem->sem_count++;
    KASSERT(sem->sem_count > 0);
    wchan_wakeone(sem->sem_wchan, &sem->sem_lock);
    spinlock_release(&sem->sem_lock);
}
```

- 1) a cosa serve lo spinlock (in entrambe le funzioni)?
- 2) perché la P contiene un ciclo while anziché un if (sem->sem_count == 0)?
- 3) Perché la wchan_sleep riceve come parametro lo spinlock?
- 4) E' possibile che la chiamata alla wchan_wakeone svegli più di un thread in attesa su

- 1) lo spinlock serve a garantire l'accesso in mutua esclusione al contatore sem->sem_count. Va aggiunto che lo spinlock è necessario per poter utilizzare il wait channel (proprio perché si suppone che il wait channel sia usato in corrispondenza a una sezione critica).
- 2) Perché la semantica "Mesa" con cui sono implementati i wait channel, non garantisce che, pur essendo la condizione sem->sem_count > 0 vera al momento in cui viene eseguita la V, lo sia ancora quando il thread in attesa esce dalla wchan_sleep.
- 3) Per poterlo rilasciare, e successivamente riacquisirlo prima dell'uscita dalla wchan_sleep.
- 4) NO. La wchan_wakeone può svegliare solo un thread in attesa. Per svegliare più thread si dovrebbe usare wchan_wakeall.

C) Spiegare brevemente la differenza tra condition variable e wait_channel.

Si tratta di due primitive di sincronizzazione simili nel comportamento. Si potrebbe dire, in modo leggermente improprio, che un wait channel è una condition variable, che invece di usare un lock usa uno spinlock. Proprio per questo, il wchan è una primitiva utilizzabile solo all'interno del kernel. Il wchan è la primitiva fornita in OS161 per realizzare sincronizzazioni di tipo wait/signal, quindi prive di busy wait.

A) La funzione cv_wait riceve un lock come parametro perché:

- è necessario come parametro nella chiamata interna a wchan_sleep; [Sì/NO: SI]
- il thread chiamante deve essere il proprietario del lock; [Sì/NO: SI]
- il lock deve essere rilasciato e successivamente riacquisito da cv_wait; [Sì/NO: NO]
- il lock deve essere rilasciato e successivamente riacquisito da wchan_sleep. [Sì/NO: SI]

Motivazioni: _____

B) Un lock può essere implementato:

- mediante un semaforo binario, senza alcun altro elemento o requisito; [Sì/NO: NO]
- mediante un semaforo binario, insieme a un elemento o requisito aggiuntivo; [Sì/NO: SI]
- mediante una variabile condition; [Sì/NO: NO]
- mediante un wait channel. [Sì/NO: NO]

Motivazioni: _____

C) L'implementazione di lock_acquire può includere una chiamata a:

- spinlock_data_get e spinlock_data_testandset; [Sì/NO: SI]
- funzione P applicata a un semaforo; [Sì/NO: SI]
- cv_wait; [Sì/NO: NO]
- ~~wchan_sleep~~; [Sì/NO: NO]

Una condvar non serve a realizzare la mutua esclusione e richiede già l'esistenza di un lock. Permette semplicemente ai thread di attendere una condizione che si verifichi, mentre il lock protegge i dati condivisi.

A.1) è Sbagliata, è NO → wchan_sleep non riceve un normale lock, ma uno spinlock associato al wait channel. Il lock viene passato a cv_wait con una funzione diversa.

A.2) Prima di chiamare il cv_wait il thread deve possedere il lock per proteggere la condizione condivisa.

A.3) è Sbagliata, è SI → cv_wait deve rilasciare il lock, mettere il thread in attesa e riacquisire il lock prima di ritornare. Il lock deve essere rilasciato altrimenti gli altri thread non potrebbero acquisirlo e modificare la condizione per svegliare il thread.

A.4) è Sbagliata, è NO → wchan_sleep opera sullo spinlock del wait channel, non sul normale lock passato a cv_wait, che invece gestisce il rilascio / acquisizione del lock normale.

B.1) Un semaforo binario garantisce la mutua esclusione, ma non registra il proprietario, cosa necessaria per il lock (solo il thread proprietario può eseguire lock_release).

B.2) Possiamo usare un semaforo binario con aggiunta della gestione della proprietà del lock.

B.3) La condvar non serve a realizzare la mutua esclusione, richiede l'assistenza di un lock sulla condizione condivisa.

B.4) è Sbagliata, è si

C.1) sarebbe NO → sono funzioni di basso livello usate per implementare gli spinlock. Non devono essere implementate.

C.4) sarebbe SI → se il lock è implementato mediante wait channel e spinlock, quando il lock è occupato lock_acquire può mettere in attesa il thread con wchan_sleep () e poi verrà risvegliato quando il lock sarà rilasciato.

Un lock può essere implementato utilizzando:

- un wait channel, per mettere in attesa i thread;
- uno spinlock, per proteggere lo stato interno;
- una variabile che indica il proprietario.

SPINLOCK

```
void
spinlock_acquire(struct spinlock *splk)
{
    // ...

    while (1) {
        if (spinlock_data_get(&splk->splk_lock) != 0) {
            continue;
        }

        if (spinlock_data_testandset(&splk->splk_lock) != 0) {
            continue;
        }

        break;
    }

    // ...
}
```

```
void
spinlock_acquire2(struct spinlock *splk)
{
    // ...

    while (1) {
        while (spinlock_data_get(&splk->splk_lock) != 0) {
            // Attesa attiva
        }

        if (spinlock_data_testandset(&splk->splk_lock) == 0) {
            continue;
        }

        break;
    }

    // ...
}
```

Abbiamo queste 2 versioni di `spinlock_acquire()` (funzione usata per acquisire uno spinlock). Nella seconda versione VA aggiunto un BREAK per farla funzionare.

`Spinlock_data_get()` → legge solo il valore del lock (=1 → occupato)

`Spinlock_data_testandset()` → controlla e imposta il lock in modo ATOMICO

- Perché `spinlock_acquire()` utilizza sia `spinlock_data_get()` sia `spinlock_data_testandset()`, invece di chiamare soltanto una delle due? `spinlock_data_get()` viene utilizzata nel ciclo di attesa (BUSY WAIT) perché è una semplice lettura ed è meno costosa. Quando il lock sembra libero, è però necessario utilizzare `spinlock_data_testandset()` per acquisirlo atomicamente. La sola `get()` non garantirebbe la mutua esclusione, perché un'altra CPU potrebbe acquisire il lock tra la lettura e la successiva modifica.
- `spinlock_acquire2()` è equivalente a `spinlock_acquire()`? In caso contrario, come può essere corretta? NO → Quando `testandset()` restituisce 0, significa che il lock è stato acquisito. La prima funzione esce correttamente dal ciclo, mentre la seconda esegue `continue` e successivamente rimane in attesa sul lock che ha appena acquisito. Per correggerla, il `continue` eseguito quando `testandset()` restituisce 0 deve essere sostituito con `break`, altrimenti avremmo un loop infinito.

SYS_EXIT, SYS_WAITPID, PROC_DESTROY()

Funzioni sys__exit e sys_waitpid

Sono già presenti nella versione base di OS/161? (SÌ/NO) NO

Sono chiamate di sistema che possono essere invocate dai processi utente? (SÌ/NO) NO

Identificazione dei processi mediante PID

Nel laboratorio 4, un processo deve essere identificato mediante un PID.

Un PID è un puntatore? (SÌ/NO) NO

Il PID viene associato a un processo durante il bootstrap? (SÌ/NO) SÌ

Terminazione e attesa

La funzione common_prog va in crash se waitpid, o funzioni correlate, non vengono implementate e utilizzate?
(SÌ/NO) SÌ

La funzione proc_destroy viene chiamata all'interno di:

chiamata di sistema exit? (SÌ/NO) SÌ

chiamata di sistema waitpid? (SÌ/NO) NO

funzione thread_exit? (SÌ/NO) NO

- Sys_getpid è necessaria per implementare sys_waitpid → FALSO. Waitpid serve per aspettare la terminazione di un ALTRO PROCESSO, il PID del processo da attendere gli viene direttamente passato come PARAMETRO: pid_t sys_waitpid(pid_t pid, userptr_t statusp, int options);
- sys_getpid è necessaria per implementare la system call getpid e viene chiamata direttamente da un processo utente. FALSO → Non viene chiamata direttamente dal processo utente perché è una funzione del kernel. Viene però utilizzata per implementare la system call getpid().
- sys_getpid implementa la system call getpid e viene chiamata dal kernel nella funzione syscall. VERO
- Sys_getpid() restituisce il PID del processo corrente? VERO
- sys_getpid viene chiamata per ottenere il PID di un altro processo, cioè quello di cui si attende la terminazione. FALSO
- sys_getpid viene chiamata da sys__exit per conoscere il PID del processo da segnalare. FALSO → sys__exit già opera sul processo corrente (curproc), non deve recuperare il suo pid con sys_getpid().
- Per proc_wait() dobbiamo avere questo ordine:
- 1. Acquisizione del lock → lock_acquire() (prima di chiamare cv_wait, il thread deve già possedere il lock associato alla condvar. Poi cv_wait() automaticamente rilascia il lock, mette il thread in attesa, quando riceve il segnale, risveglia il thread, riacquisisce il lock e ritorna al chiamante).
- 2. Attesa sulla condition variable → cv_wait()
- 3. Risveglio e riacquisizione automatica del lock → implementato direttamente da cv_wait quando arriva segnale
- 4. Lettura dello stato del processo terminato → return_status
- 5. Rilascio del lock → lock_release()

- 6. Distruzione del processo terminato → `proc_destroy()`
- 7. Restituzione dello stato

Dopo `cv_wait()` NON possiamo fare:

```
if (return_status < 0)

    return return_status;
```

Perché stiamo ritornando senza rilasciare il lock → PROBLEMA (deadlock)

Si può benissimo eliminare il controllo.

`Proc_destroy()` NON deve distruggere `curproc`, ma `proc`, ossia il processo passato come parametro!!!

- Perché il processo terminato non può distruggersi subito?

Quando un processo chiama `exit()`, il padre potrebbe chiamare successivamente: `waitpid(pid, &status, 0);`

Il padre deve recuperare ancora: PID figlio, exit status, info figlio terminato.

Se il figlio chiamasse immediatamente: `proc_destroy(curproc);` queste info verrebbero eliminate prima che il padre possa leggerle. Il processo viene quindi lasciato temporaneamente come ZOMBIE.

- Come si passa da PID a processo e viceversa?

DA PROCESSO A PID → inseriamo il PID nella struct `proc`.

```
pid_t
proc_to_pid(struct proc *p)
{
    return p->p_pid;
}

struct proc {
    pid_t p_pid;
};
```

DA PID A PROCESSO → il PID è un numero, quindi per trovare il relativo processo serve una tabella globale: `struct proc *pid_table[PID_MAX];`
L'indice è il PID e il contenuto è il puntatore al processo

```
struct proc *
proc_from_pid(pid_t pid)
{
    struct proc *p;

    lock_acquire(pid_table_lock);
    p = pid_table[pid];
    lock_release(pid_table_lock);

    return p;
}
```

lock protegge la tabella.

Per implementare `waitpid` servono quindi:

- PID processo
- PID padre
- Exit status
- Indicazione stato processo: se exited o zombie
- Tabella PID → processo
- Lock

GESTIONE DELLA CONSOLE

```
void putch_intr(struct con_softc *cs, int ch) {
    P(cs->cs_wsem);
    cs->cs_send(cs->cs_devdata, ch);
}

void con_start(void *vcs) {
    struct con_softc *cs = vcs;
    V(cs->cs_wsem);
}
```

- Siccome putch_intr è chiamata da putch(int ch), si tratta di un output. Perché putch_intr aspetta con P invece di segnalare con V?

Il semaforo cs_wsem indica la disponibilità della console a trasmettere un nuovo carattere. La console può trasmettere un solo carattere alla volta. Con P() controlliamo se la console è libera e se è occupata il thread viene sospeso. Quando diventa libera il thread invia il carattere con cs_send(). Quando l'hw ha terminato la trasmissione, segnala il completamento con V(), in modo tale che possa essere inviato un altro carattere.

Il semaforo viene inizializzato a 1 perché, all'inizio, la console è libera.

- Come viene definito cs_wsem?

Nella struttura con_softc viene definito come : struct semaphore *cs_wsem;

Viene creato così: cs->cs_wsem = sem_create("console write", 1);

Il valore iniziale 1 significa che è possibile avviare immediatamente la prima scrittura.

- Come si potrebbe usare una condition variable?

Una condition variable non conserva uno stato. Serve quindi una variabile booleana che indichi se la console è disponibile:

```
struct lock *cs_wlock;
```

```
struct cv *cs_wcv;
```

```
bool cs_wready;
```

Il produttore aspetta finché la console non è libera:

```

lock_acquire(cs->cs_wlock);

while (!cs->cs_wready) {
    cv_wait(cs->cs_wcv, cs->cs_wlock);
}

cs->cs_wready = false;
cs->cs_send(cs->cs_devdata, ch);

lock_release(cs->cs_wlock);

```

Quando l'output termina:

```

lock_acquire(cs->cs_wlock);

cs->cs_wready = true;
cv_signal(cs->cs_wcv, cs->cs_wlock);

lock_release(cs->cs_wlock);

```

cs_wready contiene la condizione; la condition variable serve solamente a sospendere e risvegliare i thread. Tuttavia, in OS/161 con_start() può essere eseguita in un interrupt handler. Un interrupt handler non può sospendersi e quindi non dovrebbe acquisire un normale lock. Per questo il semaforo utilizzato dal codice originale è più adatto.

STRUCT SPINLOCK

```

struct spinlock {
    struct spinlock_data splk_lock;
};

```

Spinlock_data contiene il dato manipolato atomicamente dall'hw. Potremmo anche aggiungere il proprietario (quale CPU possiede lo spinlock):

```

struct spinlock {
    struct spinlock_data splk_lock;
    struct cpu *splk_holder;
};

```

PROCESSO UTENTE

- lo switchframe di un processo si trova nel suo kernel stack ed è allocato dinamicamente mediante kmalloc. FALSO → Kmalloc() alloca memoria nello heap del kernel (parte dinamica), invece lo switchframe è ricavato all'interno del KERNEL STACK DEL THREAD, già esistente
- switchframe_switch è implementata in assembly e cambia contesto salvando quello di un thread e ripristinando quello di un altro. VERO (si lavora direttamente con i registri della CPU, quindi deve essere implementato in assembly).

- lo user stack non appartiene allo spazio di indirizzamento del processo, che comprende solamente codice e dati. FALSO
- in dumbvm, lo stack utente di un processo ha una dimensione fissa di 18 pagine. VERO
- enter_new_process avvia un processo utente creando il contesto necessario e simulando un ritorno da eccezione. VERO. Prima della chiamata il kernel prepara: entypoint del programma, stack pointer utente, argomenti (a0-a3), address space. Enter_new_process() configura i registri della CPU come se stesse tornando da una trap/eccezione. Quindi pc impostato su entypoint, stack pointer impostato su user stack, modalità di esecuzione in USER MODE, registri argomenti preparati. E poi viene eseguito il meccanismo equivalente ad un "return from exception".
- enter_new_process attende che il processo termini e restituisce il suo exit status. FALSO: trasferisce l'esecuzione dal kernel al nuovo programma utente, quindi non ritorna al proprio chiamante. L'exit status viene gestito da: proc_wait, sys__exit, sys_waitpid.
- durante una read(fd, buf, nb), buf è la sorgente e fd è la destinazione. FALSO: fd è la sorgente da cui leggere, buf è l'area di memoria in cui copiare i dati
- quando un processo utente esegue read(fd, buf, nb), buf può puntare a un buffer del kernel se è attivo il kernel buffering. FALSO: la chiamata è effettuata da un programma utente, quindi buf deve SEMPRE rappresentare un indirizzo appartenente allo spazio utente (userptr_t buf).
- l'handler delle system call MIPS di OS/161 riceve i dati necessari nel trapframe e ha quindi un solo parametro, il puntatore al trapframe. VERO
- quando syscall termina, esegue un'operazione V su un semaforo sul quale il processo utente è in attesa. FALSO
- Uno switchframe memorizza il contesto del processo da ripristinare al ritorno da una chiamata di sistema. NO → lo switchframe è usato durante un context switch tra thread.
- Un trapframe viene allocato nello stack kernel di un processo utente e utilizzato per avviare il processo tramite mips_usermode(). SI
- Un processo utente viene inizialmente avviato come thread del kernel, eseguendo cmd_progthread e runprogram. SI
- thread_make_runnable, chiamata da thread_fork, manda immediatamente il nuovo thread in esecuzione sulla CPU non appena lo switchframe è pronto. NO → La funzione rende il thread runnable, cioè pronto per essere scelto dallo scheduler, viene messo nella RUN QUEUE.
- In OS/161, la struct proc di un processo multithread contiene un puntatore alla lista dei thread attivi. NO → contiene il numero di thread appartenenti
- Lo spazio degli indirizzi di un processo deve essere liberato chiamando as_destroy() nell'implementazione kernel di _exit(), cioè sys__exit(). SI →

quando chiamiamo `sys__exit()` avviene la separazione dell'address space dal processo, `as_destroy()` rilascia le risorse e poi `thread_exit()`.

- La struct `proc` associata a un processo utente contiene direttamente il puntatore allo stack utente. NO → contiene il puntatore all'address space: `struct addrspace *p_addrspace`; Lo stack utente appartiene allo spazio di indirizzamento del processo. Lo stack pointer corrente viene inoltre conservato nel registro appropriato quando si entra o si ritorna in modalità utente.
- `Enter_new_process()` → `enter_new_process()` non ritorna al chiamante perché il thread kernel passa in modalità utente e comincia a eseguire le istruzioni del processo utente. SI → La funzione prepara i registri necessari come : SP, PC, argc, argv e poi entra in modalità utente, se ritornasse sarebbe considerato un errore.
- Un processo diventa zombie dopo aver terminato, ma può essere ripreso mediante la chiamata di sistema `restartpid()`. NO → Processo zombie termina definitivamente l'esecuzione, rimangono solo alcune info necessarie al padre come PID e `EXIT_STATUS`. Il padre poi usa `waitpid()` per leggere lo stato di terminazione e consentire la rimozione definitiva del processo.
- Un programma utente può chiamare direttamente `thread_fork()` per creare un processo utente multithread. NO → è una funzione interna del kernel e non può essere chiamata in modalità user.
-
- Come sono collegati processo e thread?

Ogni thread contiene nella sua struct:

`struct proc *t_proc`; □ puntatore che indica il processo al quale appartiene il thread. La struct `proc` invece contiene: `unsigned p_numthreads`; (indica quanti thread appartengono al processo)

- Dal processo possiamo trovare tutti i suoi thread? Nella versione base NO. Da un thread possiamo ottenere facilmente il processo con `thread->t_proc`, ma da un processo conosciamo solo il numero dei suoi thread. Non abbiamo i loro indirizzi.

Per risolvere il problema nella struct `proc` dovremmo aggiungere una lista

/array:

`struct thread **p_threads;`

`struct lock *p_threads_lock;`

A quel punto possiamo attrarre l'elenco:

```

void
ThreadNames(struct proc *p)
{
    lock_acquire(p->p_threads_lock);

    for (unsigned i = 0; i < p->p_numthreads; i++) {
        kprintf("%s\n", p->p_threads[i]->t_name);
    }

    lock_release(p->p_threads_lock);
}

```

Il lock è necessario perché mentre percorriamo l'elenco un altro thread potrebbe essere aggiunto o terminare.

T_context → punta allo switchframe che contiene il contesto kernel salvato durante il context switch (quando il kernel sospende un thread). Punto da cui il thread riprende dopo il context switch.

THREAD EXIT

- Distrugge la struttura del thread e restituisce il puntatore al processo padre. — FALSO → thread_exit() non restituisce nulla e non distrugge immediatamente il thread corrente. Lo pone nello stato S_ZOMBIE; sarà un altro thread a distruggerlo successivamente.
- Decrementa il numero di thread appartenenti al processo. — VERO → Lo fa attraverso proc_remthread(curthread).
- Chiama proc_remthread() → vero → rimuove il thread dal processo e aggiorna il numero di thread associati.
- Assegna direttamente NULL al puntatore al thread nella struttura del processo. — FALSO secondo la formulazione del professore → è proc_remthread() ad assegnare t->t_proc = NULL, non thread_exit().

thread_make_runnable(newthread, false); → Il parametro booleano indica se il chiamante possiede già il lock della run queue:

- false: il lock non è già posseduto, quindi thread_make_runnable deve acquisirlo;
- true: il chiamante possiede già il lock.

Il nuovo thread verrà eseguito successivamente, quando lo scheduler lo selezionerà.

In thread_fork() normalmente si arriva a return 0. thread_make_runnable inserisce il nuovo thread nella run queue e poi restituisce il controllo al thread chiamante. Non effettua un context switch immediato.

THREAD_SWITCHM THREAD_YIELD, THREAD_CREATE, THREAD_FORK

Il thread Corrente può lasciare la CPU per diversi motivi:

- Esegue volontariamente thread_yield() quindi torna a S_READY

- Termina → passa a S_ZOMBIE
- Deve attendere un evento → passa a S_SLEEP

La funzione generale che gestisce tutti questi casi è `thread_switch(newstate, wc, lk)`, dove `newstate` indica lo stato in cui deve essere messo il thread corrente.

- 1) `thread_switch` è più generale, poiché `thread_yield` è implementata dalla chiamata `thread_switch(S_READY, NULL, NULL)`. Si → `thread_yield()` rappresenta un caso specifico di cambio contesto: il thread corrente rinuncia volontariamente alla CPU ma rimane pronto per essere eseguito → `thread_switch(S_READY, NULL, NULL)`
- 2) `thread_switch` lavora sulla ready queue, rimuovendo dalla coda un thread pronto per l'esecuzione. Si → La ready queue contiene i thread nello stato S_READY, cioè pronti a essere eseguiti ma momentaneamente senza CPU. Se il thread corrente (`cur`) deve rimanere pronto passa allo stato READY e viene inserito nella READY QUEUE. Successivamente lo scheduler lo estrae e diventa RUNNING. `thread_switch()` sistema il thread corrente secondo il nuovo stato e seleziona dalla ready queue il prossimo thread.
- 3) `thread_switch` chiama `switchframe_switch(&cur->t_context, &next->t_context)`, in cui i registri della CPU vengono salvati in `next->t_context` e ripristinati da `cur->t_context`. No → Si salva il thread che sta lasciando la CPU (`cur`) e si ripristina quello che deve riceverla (`next`).
- 4) Il prototipo di `thread_yield` è `void thread_yield(void)`, quindi la funzione può essere utilizzata solo per interrompere l'esecuzione del thread corrente e metterlo nello stato S_READY. → si → `thread_yield()` agisce soltanto su `curthread`: gli toglie temporaneamente la CPU e lo rimette READY
- 5) `thread_switch` viene chiamata da `wchan_sleep` per mettere un thread nello stato S_SLEEP, che significa attesa. Si → La funzione `wchan_sleep()` mette il thread corrente in attesa sul wait channel: `thread_switch(S_SLEEP, wc, lk)`; → il lock serve ad evitare una race condition tra il thread che si sta per addormentare e il thread che potrà eseguire il wake-up, altrimenti questo potrebbe andare perso. `wchan_sleep()` usa `thread_switch(S_SLEEP, ...)`: il thread lascia la CPU e rimane sospeso fino a un wake-up.

`Thread_create()` → costruisce e inizializza la struttura del thread

`Thread_fork()` completa la preparazione e rende il thread schedulabile.

- 1) `thread_create` alloca e inizializza semplicemente la struttura C del thread. Si → crea la nuova struct → struct thread e inizializza i suoi campi principali. non completa da sola l'avvio del thread.
- 2) `thread_fork` chiama `thread_create`. Si → `thread_fork()` utilizza `thread_create()` come primo passo per ottenere una struttura del nuovo thread. Infatti per prima cosa crea la struct thread, alloca il kernel stack, prepara il contesto iniziale,

associa la funzione che il kernel thread deve eseguire e inserisce nella ready queue pronto per schedulazione.

- 3) `thread_fork` genera un processo utente figlio. NO → Crea un nuovo thread del kernel. `Fork()` system call genera un nuovo processo utente figlio. `thread_fork()` crea un thread; la system call `fork()` crea un processo e può usare `thread_fork()` come passaggio interno.
- 4) `thread_fork` riceve come parametro un puntatore a una funzione da richiamare in caso di errore, per esempio se il thread non può essere creato. NO → La funzione passata a `thread_fork()` non è un gestore degli errori, ma è la funzione che il nuovo thread dovrà eseguire quando riceverà per la prima volta la cpu.
- 5) `thread_fork` ha un parametro `struct proc *proc`, che può essere NULL oppure un puntatore a una struct `proc` esistente. SI → Il parametro indica il processo al quale deve essere associato il nuovo thread.
- 6) Né `thread_switch`, né `thread_fork` vengono utilizzate per creare un processo. Abbiamo delle funzioni dedicate alla gestione dei processi come `proc_create()` che inizializza un struct `proc` che rappresenta un processo e contiene le sue risorse, oppure `proc_create_runprogram()`.
- 7) `Thread_switch()` non crea thread, ma gestisce il passaggio da thread corrente a prossimo thread.
- 8) Quando viene creato un nuovo kernel thread in OS/161, questo appartiene al processo che lo ha creato o può appartenere a un processo diverso? Motivare. Può appartenere ad un processo diverso da quello del creatore perché `thread_fork()` riceve esplicitamente come parametro un puntatore ad un processo: `struct proc *proc`.
- 9) Perché la funzione `thread_exit()`, nella versione base di OS/161, non può essere chiamata dopo che si è chiamata `proc_destroy()` per il processo al quale appartiene il thread? Ogni thread possiede un riferimento al processo di appartenenza (`t_proc`). Se eseguiamo prima `proc_destroy(curthread->t_proc)`, la memoria della struct `proc` verrebbe liberata e a quel punto `curthread->t_proc` continuerebbe a contenere l'indirizzo della vecchia struct, non più valido (dangling pointer). Se successivamente `thread_exit()` provasse ad accedere al processo o a rimuovere il thread dal processo, utilizzerebbe memoria già liberata. Inoltre `proc_destroy()` deve essere chiamata solo quando nessun thread appartiene + al processo: `proc->p_numthreads == 0`. Perciò il thread corrente deve prima essere separato dal processo, per esempio tramite:
`proc_remthread(curthread);`
Soltanto dopo che il processo non contiene più thread può essere distrutto.
- 10) Cosa succede se non viene chiamata la funzione `as_destroy` al termine dell'esecuzione di un processo user? Il kernel può andare in crash? Motivare. Se non viene chiamata `as_destroy()`, l'address space del processo non viene liberato correttamente. `as_destroy()` deve liberare almeno:

- i frame fisici del segmento di codice;
- i frame fisici del segmento dati;
- i frame fisici dello stack;
- la struttura struct addrspace;
- eventuali altre strutture associate alla memoria virtuale.

As_destroy(as) in dumbvm restituisce all'allocatore i frame associate alle basi fisiche (3 pbase, 3 freeppages()) e infine libera la struttura con kfree(as).

Se as_destroy() non viene chiamata, il processo termina ma i suoi frame continuano a risultare occupati: si verifica memory leak del kernel. Ma il kernel non necessariamente va immediatamente in crash. Potrebbe continuare a funzionare finché rimangono frame liberi, fino a quando la memoria non esaurisce.

Inoltre proc_destroy potrebbe richiedere che l'address space sia già stato scollegato e distrutto.

PROC_DESTROY()

- Può essere chiamata dentro sys__exit, dopo thread_exit(), se il numero di thread diventa zero. — FALSO → Dopo thread_exit() il thread non continua l'esecuzione: viene effettuato uno switch e la funzione non ritorna. Inoltre il processo non deve essere necessariamente distrutto subito, perché il padre potrebbe dover leggere lo stato d'uscita mediante waitpid().
- Non può essere chiamata prima di thread_exit(), a meno che tutti i thread siano stati scollegati e lo stato di uscita sia stato letto. — VERO → Prima di distruggere il processo, nessun thread deve ancora essere associato, lo stato di uscita deve essere raccolto quando richiesto
- Attende la terminazione del processo e poi libera la struttura del processo. — FALSO → proc_destroy() non attende, libera le risorse del processo, in modo che poi si possa distruggere.
- Legge lo stato d'uscita e lo restituisce al chiamante. — FALSO → waitpid() deve farlo, non proc_destroy()

WAITPID()

- Riceve come parametro il puntatore al processo da attendere. — FALSO → riceve il PID del processo da attendere
- **Attende un cambiamento di stato del processo; nel laboratorio viene gestita soltanto l'uscita. — VERO → dobbiamo aspettare che il figlio termini**
- Può essere chiamata da common_prog() dopo aver creato un processo user. — FALSO → common_prog() viene usata per avviare programmi dal menu del kernel, non è padre user che esegue syscall.
- Dopo il risveglio può chiamare proc_destroy(curproc). — FALSO. Deve distruggere eventualmente la struttura del processo figlio atteso, non di curproc che identifica il chiamante.
- Perché il processo non può chiamare as_destroy dopo thread_exit()? Perché thread_exit() termina definitivamente il thread e non ritorna. Di conseguenza, qualsiasi istruzione successiva, compresa as_destroy(), non verrebbe mai

eseguita. L'address space deve quindi essere distrutto prima di chiamare `thread_exit()`, oppure successivamente da un altro processo/thread incaricato della pulizia.

AS_DEFINE_REGION()

```
#define PAGE_SIZE 4096 /* size of VM page */
#define PAGE_FRAME 0xfffff000 /* mask for getting page number from addr */
...
as_define_region(struct addrspace *as, vaddr_t vaddr, size_t sz,
                 int readable, int writeable, int executable) {
    size_t npages;
    ...
    /* Align the region. First, the base... */
    sz += vaddr & ~(vaddr_t)PAGE_FRAME;
    vaddr &= PAGE_FRAME;
    /* ...and now the length. */
    sz = (sz + PAGE_SIZE - 1) & PAGE_FRAME;
    npages = sz / PAGE_SIZE;
```

```
...
if (as->as_vbase1 == 0) {
    as->as_vbase1 = vaddr;
    as->as_npages1 = npages;
    return 0;
}
...
}
```

Viene chiamata da `load_elf()`. `as_define_region` descrive una regione virtuale, ma non assegna ancora i frame fisici.

Il codice usa:

```
#define PAGE_SIZE 4096
```

```
#define PAGE_FRAME 0xfffff000
```

➔ La pag è grande $4096 = 2^{12}$ byte. Gli ultimi 12 bit dell'indirizzo rappresentano l'offset nella pagina, mentre i bit precedenti identificano la pag.

1) `as` è un puntatore a una struct `addrspace`, che viene allocata all'interno di `as_define_region` e successivamente restituita al chiamante. NO ➔

Il parametro: `struct addrspace *as` ➔ è un puntatore ad una struttura già creata precedentemente, tipicamente con `as_create()`. `As_define_region()` non crea l'address space, ne riempie alcuni campi. Per allocarlo / restituirlo il parametro dovrebbe essere: `struct addrspace **as`, oppure dovrebbe restituire direttamente: `struct addrspace *`.

2) All'avvio di un processo utente, `as_define_region()` viene chiamata tre volte: per segmento di testo, segmento dati e stack utente. SI ➔ Il significato concettuale è che devono essere definite le principali regioni virtuali del processo: segmento di testo, segmento di dati, stack utente. Solitamente `as_define_region()` viene chiamata per chiamare i segmenti caricabili dall'eseguibile (ELF: data + codice), invece `as_define_stack()` per configurare lo stack.

- 3) I segmenti di testo e dati vengono riconosciuti soltanto in base all'ordine delle chiamate: la prima chiamata riempie `as_vbase1` e `as_npages1`, mentre la seconda riempie `as_vbase2` e `as_npages2`.--> SI. Infatti con il codice abbiamo:

```
if (as->as_vbase1 == 0) {
    as->as_vbase1 = vaddr;
    as->as_npages1 = npages;
    return 0;
}
```

La funzione non riconosce il segmento dal nome. Utilizza semplicemente l'ordine delle chiamate. Se la prima regione è ancora vuota (`==0`) allora salva la regione nei campi ...1, altrimenti nei campi ...2

- 4) L'istruzione seguente calcola l'offset all'interno della pagina: `vaddr &= PAGE_FRAME`; → NO → Per calcolare l'offset dovremmo: `offset = vaddr & 0xFFF`; Invece qui stiamo facendo questo:

`vaddr: 00030123`

`PAGE_FRAME: FFFFF000`

`risultato: 00030000`

Quindi con l'AND stiamo azzerando gli ultimi 12 bit dell'indirizzo. Allinea l'indirizzo verso il basso all'inizio della pag che contiene `vaddr`.

- 5) `sz = (sz + PAGE_SIZE - 1) & PAGE_FRAME`; questa istruzione calcola la frammentazione interna nell'ultima pag? NO → arrotonda `sz` verso l'alto al multiplo successivo di 4096. Per calcolare la frammentazione dobbiamo fare: dimensione arrotondata – dimensione effettiva

- 6) Dati :

`vaddr = 0x30123`

`sz = 8192 byte`

`OFFSET = 0x123 = 291`, quindi non è allineato.

Con questa istruzione: `sz += vaddr & ~(vaddr_t)PAGE_FRAME`; calcoliamo :

`sz=8192+291=8483 byte` (sommiamo l'offset alla dimensione `sz`).

Con questa: `vaddr &= PAGE_FRAME`; allineiamo `vaddr`

`0x30123 & 0xFFFFF000 = 0x30000`

Questo è il valore assegnato a `as->as_vbase1`

Invece il valore di `as->as_npages1`?

`Sz = 8483`. Quindi:

$2 * 4096 = 8192$ (2 pag non bastano)

$3 * 4096 = 12288$ (ok)

Dopo l'arrotondamento `sz = 12288 byte`

`Npages = sz / PAGE_SIZE = 3`

- Frammentazione interna = $4096 - (0x123 + 1) = 3804$ byte
+1 perché consideriamo anche la posizione con offset 0.

- La funzione alloca memoria fisica per i segmenti di codice e dati, ma non per lo stack. NO → non alloca memoria fisica
- La funzione definisce semplicemente, e memorizza nei campi dati appropriati di una struttura `addrspace`, l'indirizzo di partenza e la dimensione di un segmento per ogni chiamata di funzione. SI
- L'istruzione `vaddr &= PAGE_FRAME` cancella i 12 bit meno significativi di un indirizzo fisico. SI
- L'istruzione: `sz += vaddr & ~(vaddr_t)PAGE_FRAME`; può essere sostituita da:
`sz += vaddr % 4096`;

`~PAGE_FRAME = 0x00000fff`

`~PAGE_FRAME = 0x00000fff` → conserviamo solo i bit inferiori, ossia l'offset.

ALGORITMI DI SOSTITUZIONE DELLE PAG

WORKING SET

Non è difficile da implementare perché è difficile scegliere un buon valore di delta. La scelta di delta influenza però le prestazioni (un delta troppo grande conserva pag poco utilizzate). E' UNA politica di sostituzione delle pag a dimensione FISSA.

E' difficile da implementare perché :

- Il resident set deve essere aggiornato anche quando non si verifica un PF.
- Deve tenere traccia dell'istante dell'ultimo accesso per ciascuna pag.

NON richiede di conservare una lista di tutti gli istanti di accesso per ciascuna pag.

Domanda: il **thrashing** può verificarsi anche se la somma dei working set è inferiore alla memoria fisica disponibile? SI.

WORKING SET = insieme delle pag che un processo usa frequentemente in un determinato intervallo di tempo.

Se:

$$\sum W S_i < \text{memoria fisica disponibile}$$

significa che, teoricamente, la memoria è sufficiente per contenere tutte le pagine utilizzate attivamente.

Se però utilizziamo un algoritmo di sostituzione inadeguato, non ottimo, potrebbe continuamente eliminare pag che serviranno a breve. Quindi il processo passa + tempo a trasferire le pag tra disco e RAM (thrashing).

SOSTITUZIONE GLOBALE E LOCALE

Con la sostituzione locale, un processo con molti page fault può ridurre la memoria disponibile agli altri processi? NO. Con la SOSTITUZIONE LOCALE ogni processo possiede un insieme di frame assegnati. Se il processo A subisce un page fault può sostituire solamente una pag contenuta nei frame

1,2,3, non può prendere i frame del processo B (4,5,6). Quindi se ha molti page fault, può sostituire continuamente solo le proprie pag e le prestazioni peggiorano (la memoria assegnata a B non cambia). Con la SOSTITUZIONE GLOBALE invece A potrebbe scegliere come vittima una pag di B e quindi sottrargli un frame (rendere – disponibile della memoria per B).

La sostituzione locale garantisce una > indipendenza dagli altri processi in esecuzione perché il processo può sostituire solo le proprie pag.

Per ottenere un miglior throughput però dobbiamo usare la politica GLOBAL perché permette di distribuire dinamicamente i frame, assegnandone di più ai processi che ne hanno maggiormente bisogno. Questo va a ridurre il numero di PF complessivi e aumenta il throughput (numero di processi completati nell'unità di tempo). Aumenta anche l'interferenza tra processi.

ALLOCAZIONE EQUAL /PROPORTIONAL

- EQUAL→ assegna lo stesso numero di frame a ogni processo
- PROPORTIONAL→ assegna i frame in proporzione alla dimensione del processo.

Con il WORKING SET non si usa né EQUAL, né PROPORTIONAL perché assegna dinamicamente i frame in base alle pag utilizzare recentemente dal processo, non in base alla dimensione.